Valutazione di una Soluzione Obiettivi

- Discutere i principali criteri di qualità dell'efficacia di una soluzione, inclusi:
    - la correttezza tramite metodi empirici;
    - l'efficienza esaminando le complessità di tempo e spazio;
    - l'eleganza, in particolare la massimizzazione dell'efficacia e della semplicità;
    - l'usabilità secondo i criteri standard del settore.
- Spiegare l'importanza dei compromessi riguardo ai criteri di qualità.

Valutazione della Soluzione Ce l'hai fatta. Hai analizzato il problema, lo hai suddiviso in parti, hai elaborato una soluzione di design, che poi hai implementato, testato e corretto. Ora hai qualcosa che funziona. Quindi, hai finito, giusto? Beh, non proprio. Il lavoro non è finito solo perché hai una soluzione. Prima di poter veramente concludere, devi assicurarti di aver prodotto una buona soluzione. Devi valutarne la qualità. Ci sono molti aspetti della qualità. Valutare una soluzione implica porre diverse domande fondamentali, ognuna rivolta a un aspetto specifico. Le domande importanti sulla soluzione includono:

- È corretta? Risolve realmente il problema che ti eri prefissato di risolvere?
- È efficiente? Utilizza le risorse in modo ragionevole?
- È elegante? È semplice ma efficace?
- È utilizzabile? Offre un modo soddisfacente per il pubblico di destinazione di usarla?
- **È CORRETTA?** La domanda più importante da porre sulla tua soluzione è: risolve effettivamente il problema originale che intendevi risolvere? In altre parole, è una soluzione corretta? Se non lo è, allora rende superflue tutte le altre misure di qualità. Non importa quanto sia veloce, intelligente o accattivante la tua soluzione; se non risolve il problema, allora è errata e devi apportare modifiche.
- L'approccio prudente è assumere che un programma sia errato finché non si dimostri il contrario. Praticamente, questo ha senso; dopo tutto, una soluzione raramente è corretta prima che vengano effettuati test e debug. Filosoficamente, è anche la cosa giusta da fare. Esistono infinite possibili soluzioni errate; ce ne sono relativamente poche corrette. È lo stesso approccio scettico utilizzato in campi come la scienza, la medicina e il diritto, dove l'onere della prova grava su coloro che fanno affermazioni positive, per proteggersi contro i falsi positivi.
- Tecnicamente parlando, la correttezza di un programma è una questione profondamente teorica in informatica. Negli ambienti accademici, parlare di dimostrare la correttezza di un programma si riferisce di solito alla prova matematica. Una tale prova, se riuscita, stabilisce la correttezza di un programma come un fatto incontrovertibile. Tuttavia, i professionisti al di fuori dell'ambito accademico raramente adottano questo approccio costoso quando non si occupano di sistemi critici.
- La maggior parte delle volte, dimostrare che un programma è corretto si fa empiricamente eseguendo una serie di test, un argomento trattato nel Capitolo 5. All'inizio di un processo di sviluppo, un problema è specificato in un certo modo. Questa specifica si riflette nel piano di test (vedi un esempio di piano nel Figura 6.1), che riformula ogni requisito come un test. I test eseguono ogni aspetto della soluzione e verificano che il comportamento risultante soddisfi la specifica.

- **Figura 6.1 Estratto da un piano di test per un distributore automatico**

Test	Oggetto del Test	Descrizione del Test	Risultato Atteso	Risultato Effettivo
1	Inserire slot moneta	Inserire una moneta accettabile da 50 centesimi	Moneta accettata. ‘0.50’ appare sul display.	Comportato come previsto.
2	Inserire slot moneta	Inserire una moneta straniera non accettabile.	Moneta rifiutata. ‘Valuta non riconosciuta’ appare sul display.	Fallito. La moneta è stata rifiutata ma nessun messaggio è apparso.
3	Tastiera	Inserire una selezione valida ‘15’.	‘Hai selezionato cioccolata calda’ appare sul display.	Comportato come previsto.
4	Tastiera	Inserire una selezione non valida ‘99’.	‘Selezione non valida, riprova’ appare sul display.	Comportato come previsto.

- A differenza dell'approccio matematico, che mira a dimostrare che una soluzione è corretta, l'obiettivo di un approccio di test empirico è mostrare che la soluzione è errata. Questo è più facile perché basta un solo test fallito per dimostrarlo. Solitamente è infattibile testare ogni possibile valore che potrebbe essere inserito in una soluzione (e ogni combinazione di essi), quindi ogni test superato aggiunge un ulteriore pezzo di prova a sostegno.

Altre prove a sostegno rafforzano le tue affermazioni sulla correttezza della soluzione. Di conseguenza, mostrare la correttezza tramite test non è una questione binaria di vero o falso. Piuttosto, i test offrono un quadro sfumato della correttezza. Questo insieme di livelli ti permette di graduare la correttezza della tua soluzione, dal meno al più convincente:

1. La soluzione non contiene errori di sintassi o operazioni non valide che possono essere rilevate automaticamente.
2. La soluzione fornisce la risposta corretta per un certo insieme di dati di test.
3. La soluzione fornisce la risposta corretta per dati di test che sono tipici o casuali.
4. La soluzione fornisce la risposta corretta per dati di test che sono deliberatamente difficili da elaborare.
5. La soluzione fornisce la risposta corretta per tutti i dati di test possibili rispetto alla specifica del problema originale.
6. Come il livello 5, eccetto che una risposta corretta (o ragionevole) viene data anche per dati di test errati.
7. Per ogni possibile input, la soluzione fornisce risposte corrette o ragionevoli.

Ridurre il numero di test falliti non rafforza la tua affermazione sulla correttezza di una soluzione. La presenza anche di un solo test fallito mostra che la tua soluzione è errata. In altre parole:

- Nessun test superato e nessun test fallito significa che non hai prove della correttezza.

- Alcuni test superati e nessun test fallito significa che hai alcune prove di una soluzione corretta.
- Alcuni test superati e almeno un test fallito significa che hai una soluzione errata.

La correttezza mostra solo che la soluzione risolve il problema. Non importa se è lenta, inefficiente, troppo complicata o se semplicemente a qualcuno non piace la soluzione, questo non influisce sulla correttezza.

È EFFICIENTE? Ogni algoritmo richiede una certa quantità di risorse per svolgere il suo lavoro. Diversi algoritmi, anche quelli che risolvono lo stesso problema, possono comportarsi in modo diverso in termini di efficienza. Questa performance è solitamente misurata in termini di tempo e spazio.

- Tempo: la durata del tempo di esecuzione di un algoritmo, dall'inizio alla fine. La durata può essere misurata come il numero di passaggi effettuati durante l'esecuzione.
- Spazio: la quantità di memoria di archiviazione richiesta da un algoritmo per svolgere il suo lavoro.

La questione dell'efficienza di un algoritmo si riduce ai suoi requisiti di tempo e spazio. Se, quando analizzi un algoritmo, trovi che il suo uso delle risorse è accettabile, allora può essere considerato efficiente.

Ma la sua efficienza può davvero essere misurata in modo affidabile in un modo che possa essere confrontato equamente con quello di altri algoritmi? A prima vista potrebbe sembrare di no. Dopotutto, gli algoritmi operano su dati di input e si comportano in modo diverso a seconda della dimensione dei dati di input che ricevono. Immagina un algoritmo che somma un elenco di numeri: il tempo di elaborazione aumenta quanto più numerosi sono i numeri nell'elenco.

Classe di complessità: un profilo che ti dice come si comporteranno in generale gli algoritmi che ne fanno parte.

Questo è tutto vero, ma l'informatica può misurare l'efficienza degli algoritmi perché esamina il calcolo in generale. Fornisce principi generali sull'efficienza che puoi applicare alla tua situazione specifica. L'informatica non classifica ogni possibile algoritmo immaginabile in termini di efficienza. Invece, fornisce un insieme relativamente piccolo di classi di complessità, nelle quali ogni algoritmo può essere collocato. Considera anche il fatto che la dimensione dei dati di input può variare.

Valuta l'efficienza del tuo algoritmo analizzando il suo comportamento e associandolo a una di queste categorie.

Ogni categoria descrive come si comportano i suoi algoritmi nel caso peggiore. In altre parole, quale è la quantità massima di risorse che un algoritmo potrebbe utilizzare. Poiché la performance di un algoritmo dipende dai dati di input, l'uso delle risorse è una funzione della dimensione dei dati di input. La Tabella 6.1 mostra alcune categorie di esempio e la loro efficienza in termini di tempo.

Table 6.1 Sample of common complexity classes

Name	Running time	Comments
Constant	O(1)	Doesn't necessarily take 1 time unit, rather it takes a fixed number of time units. An example is determining if a number is odd or even.
Logarithmic	O(*logN*)	As the input size grows, the running time increases at an ever-smaller rate. Divide-and-conquer approaches often perform like this.
Linear	O(*N*)	This is the best possible performance of any algorithm that must go through the entire input sequentially.
Quadratic	O(N^2)	The running time of algorithms in this class quadruples when the input size doubles.
Factorial	O(*N*!)	Algorithms in this class are often brute force and considered too slow for practical use.

Tabella 6.1 Esempio di classi di complessità comuni

- **Nome**
- **Tempo di esecuzione**
- **Commenti**
- Costante
- Logaritmica
- Lineare
- Quadratica
- Fattoriale
- O(1)
- O(logN)
- O(N)
- O(N^2)
- O(N!)

Non necessariamente richiede 1 unità di tempo, piuttosto impiega un numero fisso di unità di tempo. Un esempio è determinare se un numero è pari o dispari.

Man mano che la dimensione dell'input cresce, il tempo di esecuzione aumenta a un ritmo sempre più piccolo. Approcci di tipo divide-et-impera spesso si comportano così.

Questa è la migliore performance possibile di qualsiasi algoritmo che deve passare attraverso tutto l'input sequenzialmente.

Il tempo di esecuzione degli algoritmi in questa classe quadruplica quando la dimensione dell'input raddoppia.

Gli algoritmi in questa classe sono spesso forza bruta e considerati troppo lenti per l'uso pratico.

Nella Tabella 6.1, il tempo di esecuzione di ciascuna categoria è espresso in una notazione particolare chiamata ironicamente "Big-O" (Grande O) per il suo uso di una grande lettera O. Il termine tra parentesi indica quanto tempo generalmente richiedono gli algoritmi di questa classe per elaborare l'input. "N" rappresenta il numero di elementi contenuti nell'input.

Ecco alcuni esempi approssimativi. Un algoritmo che esegue un passaggio (o un insieme di passaggi) per ogni elemento è classificato come O(N). Si può dire che impiega 1 unità di tempo per elaborare un input con solo 1 elemento, o impiega 5 unità di tempo per elaborare 5 elementi. In termini più generali, impiega N unità di tempo per elaborare N elementi. Come altro esempio, un algoritmo che esegue N passaggi per ciascun elemento, ovvero N × N, è classificato come O(N^2). Impiega 4 unità di tempo per elaborare 2 elementi o 16 unità di tempo per elaborare 4 elementi. Più in generale, impiega N^2 unità di tempo per elaborare N elementi.

Un esempio più concreto spiegherà meglio le classi di complessità. **Esempio: Ricerca Lineare** Un algoritmo di ricerca lineare prende due parametri di input: un elenco di elementi di dimensione N e un ulteriore elemento che chiameremo elemento di ricerca (gli elementi potrebbero essere numeri, pezzi di testo o altro). Il suo obiettivo è accertare se l'elenco contiene l'elemento di ricerca o no. L'algoritmo funziona cercando nell'elenco dall'inizio alla fine, confrontando l'elemento di ricerca con ogni elemento nell'elenco man mano che procede. Non appena un elemento dell'elenco e l'elemento di ricerca risultano uguali, l'algoritmo restituisce "vero" e si ferma immediatamente. Se l'algoritmo raggiunge la fine dell'elenco senza trovare una corrispondenza, restituisce "falso". A quale categoria di complessità appartiene l'algoritmo?

- Nel miglior caso, l'elemento di ricerca corrisponde al primo elemento dell'elenco. Questo è O(1).
- In media (assumendo che ogni elemento abbia la stessa probabilità di essere cercato), l'elemento di ricerca corrisponderà da qualche parte nel mezzo dell'elenco, il che significa che la sua prestazione media è O(N/2).
- Nel caso peggiore, o l'elemento di ricerca corrisponde all'ultimo elemento dell'elenco o non viene trovato affatto. Ciò dà una prestazione di O(N).

Chiunque voglia utilizzare un algoritmo di ricerca lineare ora ha alcune informazioni da applicare. Dovrebbero avere un'idea delle dimensioni delle liste che la loro soluzione dovrà elaborare, così come quanto tempo impiega per eseguire un confronto. Sapendo che il loro algoritmo scelto ha una prestazione nel caso peggiore di O(N), possono decidere se questo è accettabile per la loro soluzione.

Come puoi vedere, non tutti gli algoritmi sono uguali in termini di prestazioni. Algoritmi con prestazioni nel caso peggiore come O(logN), O(N) e O(N^2) sono considerati efficienti perché i loro tempi di esecuzione sono generalmente accettabili anche quando N è molto grande. Algoritmi con prestazioni nel caso peggiore come O(2N), O(N^N) o O(N!) sono generalmente considerati inefficienti perché i loro tempi di esecuzione diventano inaccettabili relativamente rapidamente man mano che N cresce.

began adding the numbers one by one. Gauss, however, quickly found a more elegant solution. He realized that by pairing the numbers (1 with 100, 2 with 99, etc.), each pair summed to 101, and since there were 50 pairs, the total was 5050. This method was not only quick but beautifully simple and effective, illustrating the concept of elegance in mathematics.

Eleganza, è davvero così?

Finora abbiamo visto che la valutazione è un'impresa oggettiva. Una soluzione software non è un'opera d'arte; è una cosa funzionale, proprio come un ponte, una teoria scientifica o una dimostrazione matematica. Pertanto, la sua correttezza ed efficienza non sono una questione di opinione.

Tuttavia, alcuni aspetti della valutazione rendono le cose un po' meno chiare in questo senso. Uno di questi aspetti è l'eleganza, qualcosa che potresti associare di più alle attività artistiche. Due soluzioni diverse potrebbero entrambe risolvere un problema, ma potrebbero essere giudicate diversamente per l'eleganza dei rispettivi approcci. Questo si applica non solo al software; è vero anche per altre discipline "funzionali" come l'ingegneria, la scienza e la matematica.

Parlando in termini generali, l'eleganza massimizza sia l'efficacia che la semplicità allo stesso tempo.

Per un ingegnere, qualcosa raggiunge l'eleganza se esegue qualcosa di molto utile con poche parti semplici. Potrebbe anche essere non ovvio e risolvere diversi problemi contemporaneamente. Come esempio, considera il tubo che drena l'acqua dal lavandino in cucina o in bagno. È collegato a uno scarico fognario e quindi rischia di far risalire cattivi odori lungo il tubo e nella stanza. Potremmo prevenire questo in molti modi. Uno sarebbe attaccare una sorta di filtro profumato all'interno del tubo, ma questo si intasa facilmente e richiederebbe pulizia e sostituzione regolari. Oppure forse una valvola di non ritorno che si apre solo in una direzione per permettere all'acqua di uscire? Non male, ma richiederebbe cerniere che rischiano di arrugginirsi. La soluzione effettiva è una trappola. Questo permette all'acqua di raccogliersi (come effetto collaterale dell'uso normale), che poi blocca qualsiasi aria che risalga dallo scarico fognario. È una soluzione incredibilmente semplice, non ovvia che funziona automaticamente e richiede poca o nessuna manutenzione (vedi Figura 6.2).

Per uno scienziato, una teoria elegante spiega molto con molto poco. Teorie come la seconda legge di Newton (F = ma) o l'equivalenza massa-energia di Einstein (E = mc^2) riducono fenomeni enormemente complessi a equazioni molto semplici. Le stesse equazioni possono essere applicate a qualcosa di piccolo come gli atomi o grande come una galassia. Servono anche come base per innumerevoli altre teorie.

Una dimostrazione matematica è elegante se è una soluzione semplice ed efficace a un problema non banale. Un esempio di eleganza matematica proviene da una storia (forse apocrifa) che vede protagonista il matematico Carl Friedrich Gauss. Un giorno, durante la sua giovinezza, il suo insegnante di matematica diede agli studenti della sua classe un problema: calcolare la somma degli interi da 1 a 100. Gauss trovò rapidamente una soluzione più elegante, realizzando che accoppiando i numeri (1 con 100, 2 con 99, ecc.), ogni coppia sommava 101 e, poiché c'erano 50 coppie, il totale era 5050. Questo metodo non era solo rapido ma anche meravigliosamente semplice ed efficace, illustrando il concetto di eleganza in matematica.

Ha iniziato un totale cumulativo e ha cominciato ad aggiungere i numeri uno alla volta (1 + 2 = 3, 3 + 3 = 6, 6 + 4 = 10 e così via). Tutti gli studenti tranne Gauss, ovvero. Lui notò un

modello nel problema: 1 + 100 = 101, 2 + 99 = 101, 3 + 98 = 101 e così via, fino a 50 + 51 = 101. Questo modello mostrava che quei 100 numeri potevano essere raggruppati in 50 coppie, ognuna delle quali totalizzava 101. Invece di eseguire 100 somme, a Gauss bastava eseguire una singola moltiplicazione (50 × 101) per ottenere la stessa risposta.

L'idea di eleganza del matematico è forse quella più vicina a quella di un informatico. Se un programmatore dovesse codificare queste due soluzioni per un computer, apparirebbero più o meno così:

La soluzione ovvia:

- Imposta total = 0;
- Per ogni numero, n, tra 1 e 100:
    - Imposta total = total + n.

La soluzione più elegante di Gauss:

- Imposta total = 50 x 101.

La soluzione di Gauss raggiunge lo stesso risultato della soluzione ovvia, ma lo fa utilizzando meno spazio (una variabile invece di due) e meno tempo (un passaggio invece di circa 100). In termini di complessità (vedi sopra), se dovessi scrivere versioni generiche di entrambi gli algoritmi per calcolare la somma dei primi N interi, la soluzione ovvia sarebbe O(N) mentre quella di Gauss sarebbe la molto più efficiente O(1).

Il semplice può essere più difficile del complesso: Devi lavorare sodo per rendere il tuo pensiero chiaro e renderlo semplice. (Jobs, 1998)

Potrebbe sembrare paradossale, ma è più difficile essere semplici che complicati. Elaborare soluzioni eleganti richiede tempo e sforzo. Richiede di arrivare all'essenza del problema e di vedere i modelli al suo interno. Richiede anche di applicare un pensiero fuori dagli schemi. Ma può essere uno sforzo molto gratificante, perché i vantaggi di un risultato elegante possono essere enormi.

Il principale svantaggio di una soluzione non ovvia è proprio questo: non è ovvia. Se qualcun altro dovesse lavorare sulla tua soluzione, ha bisogno di sapere come funziona. Chiunque guardi la soluzione ovvia sopra può facilmente capire che sta sommando i numeri da 1 a 100. Tuttavia, guardando la soluzione elegante di Gauss in isolamento, non è immediatamente chiaro cosa stia cercando di ottenere. L'autore può capirlo, ma gli altri? Pertanto, dovresti sempre assicurarti che le parti non ovvie della tua soluzione siano accompagnate da spiegazioni chiare.

Quando parlo del fatto che altre persone devono capire la tua soluzione, questo include anche te! Potresti capire le complessità oggi, ma le capirai ancora dopo aver lasciato la tua soluzione e ripreso a lavorarci su qualche settimana o mese dopo? C'è una buona possibilità che tu abbia dimenticato come funziona. Fai un favore a te stesso: scrivi sempre spiegazioni chiare di come funziona la tua soluzione, soprattutto le parti complesse.

Quando elabori soluzioni eleganti, stai attento a non essere ingegnoso solo per il gusto di esserlo. Sii sempre pronto a giustificare perché non stai prendendo l'approccio ovvio. Se non ci sono vantaggi tangibili nel tuo modo non ovvio, potresti rendere la soluzione più complessa senza una buona ragione.

È UTILIZZABILE? In definitiva, gli utenti della tua soluzione saranno persone. La tua soluzione deve quindi adattarsi agli umani e ai loro modi particolari. In altre parole, deve essere utilizzabile e dovrebbe dare agli utenti un'esperienza positiva.

Criteri come correttezza, efficienza ed eleganza descrivono una soluzione in termini propri. Tuttavia, l'usabilità misura quanto bene qualcosa può essere usata dalle persone per raggiungere i loro obiettivi. Gli umani portano un bagaglio aggiuntivo. Non siamo macchine; abbiamo desideri ed emozioni. Facciamo errori e abbiamo pazienza limitata. Le persone vogliono soluzioni facili da imparare, semplici da usare, indulgenti con la nostra fallibilità e che richiedano il minor sforzo possibile da parte nostra.

L'usabilità cattura questi desideri. Li formalizza in diversi componenti che misurano quanto sia facile e soddisfacente per le persone usare una soluzione (Nielsen, 2012). Fortunatamente, nonostante coinvolgano gli umani nell'equazione, questi componenti sono in qualche modo misurabili. Sono:

- Apprendibilità: quanto è facile per gli utenti compiere compiti di base la prima volta che incontrano la soluzione?
- Efficienza: una volta che gli utenti hanno imparato la soluzione, quanto velocemente possono eseguire i compiti?
- Memorabilità: quando gli utenti tornano alla soluzione dopo un periodo di non utilizzo, quanto facilmente possono ristabilire la competenza?
- Errori: quanti errori commettono gli utenti, quanto sono gravi questi errori e quanto facilmente possono recuperare dagli errori?
- Soddisfazione: quanto è piacevole usare il design?

Le euristiche di usabilità sono principi generali che puoi applicare quando progetti una soluzione. Se le applichi con attenzione, avrai meno lavoro da rifare nella fase di valutazione. Jakob Nielsen (1995), un esperto leader nell'usabilità, elenca dieci di queste euristiche qui: www.nngroup.com/articles/ten-usability-heuristics/

Valutare l'usabilità è in definitiva riguardo alle reazioni delle persone reali. Alla fine, dovrai coinvolgere le persone nella valutazione. Questo si fa in parte con uno studio osservazionale. Le persone rappresentative del tuo pubblico dovrebbero essere reclutate come soggetti di prova, che poi usano la soluzione per svolgere un insieme di compiti predefiniti (vedi Tabella 6.2). Mentre il soggetto è impegnato, il valutatore osserva il loro lavoro, notando dove il soggetto ha successo e dove incontra difficoltà.

Tabella 6.2 Esempio di una lista di compiti di usabilità

- **Numero Compito**
- **Descrizione**
- 1. Accedere al sistema.
- 2 . Aggiungere un indirizzo email al proprio profilo utente.
- 3. Acquistare un libro di cucina per meno di £10.
- 4. Mostrare un elenco dei propri acquisti passati.

Incoraggia il soggetto di test a essere molto esplicito durante i test. Dovrebbe pensare ad alta voce, esprimendo ciò che sta cercando di fare e quali sono i suoi pensieri attuali.

Queste sono tutte informazioni per la tua valutazione. L'evalutatore può parlare durante un test (ad esempio, per chiarire ciò che ha detto il soggetto), ma dovrebbe mantenere al minimo le sue interventi e non dovrebbe dare al soggetto suggerimenti espliciti su come utilizzare la soluzione.

Dopo che il soggetto ha completato il test, il valutatore gli fornisce un questionario. Le domande sono tipicamente ampie e non si concentrano su caratteristiche specifiche. Invece, chiedono al soggetto di valutare vari fattori su una scala da 1 (non sono per niente d'accordo) a 5 (sono completamente d'accordo). Ecco un esempio di set di domande: Valuta i seguenti aspetti su una scala da 1 a 5:

1. Nel complesso, mi è piaciuto usare questo sistema.
2. Ho trovato questo sistema facile da usare.
3. Ci ho messo troppo tempo ad abituarmi a questo sistema.
4. Ho avuto problemi a trovare le funzioni in questo sistema.
5. C'erano troppi passaggi per eseguire le funzioni in questo sistema.
6. Il sistema è stato indulgente quando ho commesso un errore.

Per ottenere maggiori informazioni, dovresti approfondire le risposte. Permetti all'utente di elaborare ogni risposta in un formato di testo libero, oppure intervistalo e chiedi le motivazioni dietro le sue risposte.

Il test di usabilità non deve necessariamente essere fatto dopo che la soluzione è completata. Potresti creare un prototipo di carta del tuo sistema e poi lasciare che i soggetti di test lo usino per darti un feedback. La regola usuale si applica: più presto trovi i problemi, più è facile risolverli.

COMPROMESSI Vari fattori cospirano per garantire che nessuna soluzione sia perfetta. Alcuni di questi fattori sono dovuti a errori commessi o alla mancanza di ore nel giorno.

Tuttavia, un altro motivo è che non sempre è possibile ottimizzare ogni aspetto di una soluzione contemporaneamente. Migliorare una soluzione sotto un certo aspetto può spesso comprometterla sotto altri. Considera l'efficienza della soluzione. È espressa in termini di tempo che la soluzione impiega e lo spazio di archiviazione che richiede. Anche se non sempre è il caso, a volte una soluzione ti costringe a scambiare una migliore performance temporale in cambio di un aumento dell'uso dello spazio – o, al contrario, accettare una soluzione più lenta come costo per una riduzione dei requisiti di spazio. Questo viene chiamato un compromesso spazio-tempo.

Come esempio illustrativo, guardiamo alla compressione delle immagini. Le immagini vengono memorizzate, inviate e renderizzate costantemente dai computer ogni giorno, su dischi, in pagine web, tramite streaming video o altrove. Perché un computer possa gestire un'immagine, essa deve essere suddivisa in pezzi discreti chiamati pixel (elementi dell'immagine), ognuno dei quali descrive una piccola parte dell'immagine.

Memorizzare tutti i pixel può sommarsi a una grande quantità di informazioni. Una tipica fotocamera da 3 megapixel (che, anche al momento della scrittura, è piuttosto comune) produce immagini con oltre 3 milioni di pixel. Ogni pixel necessita di essere memorizzato nella memoria del computer. In uno schema semplice (che useremo in questo esempio), i dati dell'immagine vengono memorizzati come una stringa di token. Ogni token è memorizzato in 1 byte, la dimensione minima indirizzabile dello spazio di memoria. Un byte

può memorizzare un singolo carattere o un numero tra 0 e 255. Il colore di un pixel è descritto da un token carattere: 'B' sta per nero, 'W' sta per bianco.

Considera l'immagine di 16 × 16 pixel di una faccina sorridente nella Figura 6.3, sulla quale è sovrapposta una griglia per mostrare esplicitamente i pixel:

La questione qui è come memorizzare i dati dell'immagine affinché possa essere nuovamente renderizzata in seguito. Una soluzione è quella di memorizzare le informazioni dei pixel in una stringa di 256 token (cioè 16 × 16), uno per descrivere ciascun pixel. Questi numeri potrebbero essere memorizzati su disco o trasmessi via internet per un utilizzo successivo. In questo caso, il rendering significa mettere i valori corretti (B o W) nel posto giusto (vedi Figura 6.4).

Questo metodo sarebbe molto veloce. Per eseguire il rendering, il computer deve solo passare attraverso ogni pixel dall'inizio alla fine, prendendo il valore successivo dalla stringa e posizionando il pixel corrispondente sullo schermo. Tuttavia, questo approccio non è ottimizzato per lo spazio. Nota che la maggior parte dell'immagine è occupata da uno sfondo bianco. Di conseguenza, i dati dell'immagine contengono molti valori ripetuti e contigui.

Quando i dati contengono molta ripetizione, possono essere compressi per occupare meno spazio. Possiamo ottimizzare la quantità di spazio necessaria per memorizzare l'immagine modificando leggermente lo schema. Invece di descrivere ogni pixel, descriviamo i pattern dei pixel. Ad esempio, quando ci sono cinque pixel neri di fila, invece di memorizzare queste informazioni come 'BBBBB' (che richiederebbe 5 byte), le memorizziamo come '5B' (che richiede 2 byte: uno per il numero e uno per il colore del pixel - vedi Figura 6.5). Utilizzando questo schema, riduciamo i requisiti di spazio. Questo aiuta particolarmente in situazioni come l'invio di immagini su internet, dove è molto vantaggioso ridurre la quantità di dati da trasmettere.

Essendo di 16 x 16 pixel, i dati dell'immagine sono sempre memorizzati usando 256 byte quando non compressi. Tuttavia, utilizzando questo schema di compressione, possiamo memorizzare la faccina sorridente usando solo 154 byte. Ciò significa che i dati dell'immagine compressa sono ridotti a circa il 60 percento delle dimensioni originali.

Tuttavia, c'è un costo che bilancia questo risparmio di spazio. La quantità di lavoro necessaria per interpretare i dati non compressi è minore di quella dei dati compressi. Un algoritmo per interpretare i dati non compressi potrebbe essere qualcosa del genere:

```
p = 0

for each token, t, in the image data:

    make pixel p the colour t

    p = p + 1
```

Mentre interpretare i dati dell'immagine compressa necessiterebbe di un algoritmo più simile a questo:

```
p = 0

for each token, t, in the image data:

    if is_number(t):

        count = t
```

```
        colour = next_token()
        for i in range(count):
            make pixel p the colour
            p = p + 1
    else:
        make pixel p the colour t
        p = p + 1
```

Questo algoritmo richiede di gestire i token che rappresentano sia numeri che colori, e di espandere i gruppi di pixel compressi nei loro singoli componenti, il che può richiedere più tempo e maggiore complessità logica.

p = 0 n = 0 per ogni token, t, nei dati dell'immagine: se t è un numero allora n = t altrimenti ripeti n volte: rendi il pixel p del colore t p = p + 1

Il processo di interpretazione dei dati di immagini compresse è più complicato e richiede più passaggi in totale, e più passaggi significano più tempo. Pertanto, abbiamo migliorato l'utilizzo dello spazio, ma a scapito dell'utilizzo del tempo durante il rendering.

Potresti aver notato altri esempi di compromessi nelle sezioni precedenti di questo libro. Il compromesso tra ingegnosità e comprensibilità, per esempio, dove incorporare trucchi potenti nella tua soluzione può renderla più difficile da comprendere. Oppure il compromesso tra usabilità e flessibilità, dove le soluzioni che sono rese più flessibili diventano meno user-friendly.

In sintesi, le tue soluzioni coinvolgeranno inevitabilmente dei compromessi. Spesso si presenterà una situazione del tipo "Voglio migliorare X, ma questo peggiorerà Y". La domanda è quindi: quanto di Y sei disposto a sacrificare per X?

Assicurati di sapere in anticipo dove ha senso ottimizzare per la tua soluzione corrente, altrimenti non avrai principi guida su quali compromessi fare. Ad esempio, se la tua soluzione implica l'invio di dati su una rete, di solito ha senso ottimizzare lo spazio di archiviazione a scapito del tempo aggiuntivo di compressione/decompressione. Questo perché trasmettere un dato richiede più tempo rispetto alla sua decodifica.

RIASSUNTO La valutazione di una soluzione comprende molti aspetti. L'aspetto più importante di una soluzione è la sua correttezza, cioè risolve effettivamente il problema originale? Oltre alla correttezza, ci sono numerosi altri aspetti relativi alla qualità di una soluzione. Quelli importanti variano da soluzione a soluzione. Questo capitolo ne ha esaminati diversi.

L'efficienza misura l'uso di tempo e spazio di una soluzione per garantire che risolva un problema con un uso accettabile delle risorse. L'eleganza determina se una soluzione massimizza sia l'efficacia che la semplicità. L'usabilità introduce un aspetto umano nella valutazione esaminando se l'utente trova la soluzione facile e soddisfacente da usare.

In questi e altri aspetti, è coinvolto un certo grado di compromesso. Una soluzione non può ottimizzare tutti gli aspetti della qualità tutto il tempo. Una parte essenziale della formazione di una soluzione è decidere quali aspetti della qualità sono più importanti e quindi quali vengono scelti per l'ottimizzazione.